

— ENGENHARIA DE EXPERIÊNCIA · 2026

AX × UX × DX

software para quem age

Como projetar, avaliar e melhorar a experiência de pessoas, desenvolvedores e agentes de IA — com evidência, não intuição.

35–45 min demo open source checklist + laboratório

"Agentes não precisam de uma interface bonita. Precisam de um caminho descobrível, acionável, verificável e seguro até o objetivo."

UX

Pessoa percebe, decide e sente a interação.

DX

Desenvolvedor constrói e opera dentro de um sistema sociotécnico.

AX

Agente interpreta contexto, escolhe ferramentas e altera estado em nome de alguém.

Experiência é relação — não atributo isolado

UX

User Experience

Ator: pessoa

Contexto: uso

Sinais: efetividade, eficiência, satisfação, acessibilidade.

DX

Developer Experience

Ator: dev/equipe

Contexto: entrega

Sinais: fluxo, feedback, colaboração, qualidade e bem-estar.

AX

Agent Experience

Ator: agente

Contexto: objetivo delegado

O mesmo detalhe técnico atravessa UX, DX e AX

Erro “500” sem causa

UX perde confiança · DX perde tempo · AX não sabe se deve corrigir, repetir ou parar.

Contrato ambíguo

UX vê inconsistência · DX cria workaround · AX escolhe parâmetros plausíveis e errados.

Feedback lento

UX abandona · DX perde fluxo · AX consome orçamento explorando.

Ação irreversível

UX sofre o dano · DX responde ao incidente · AX amplifica o impacto.

Boa DX é a **infraestrutura invisível** da AX — mas não cobre delegação, autonomia e risco de ação.

— DA CONVERSA À EXECUÇÃO

Quando o modelo ganha ferramentas, plausibilidade deixa de ser suficiente

- lê código, documentação e dados
- executa shell e pipelines
- chama APIs e serviços
- cria, altera e remove estado

MUDANÇA DE CONTRATO

" A resposta não é o resultado. O estado final é."

Por isso precisamos de oráculo independente, privilégio mínimo, sandbox e trilha auditável.

AX é a qualidade do loop completo

1. Descobrir

onde começar?

2. Compreender

qual é o contrato?

3. Planejar

qual menor ação?

4. Agir

ferramenta certa?

5. Observar

o que mudou?

6. Verificar

o objetivo foi atingido?

7. Recuperar

retry, rollback ou stop?

Affordance

README, AGENTS.md, schemas, tool descriptions.

Feedback

erros acionáveis, logs, estado consultável, testes rápidos.

Oito dimensões avaliáveis

01

Objetivo

tarefas reais + oráculo

02

Contexto

descoberta + intenção

03

Ambiente

setup + reprodutibilidade

04

Interfaces

schema + semântica

05

Feedback

estado + erros + recovery

06

Segurança

privilégio + controle

Movimento não é progresso

Não use sozinho

- linhas alteradas
- número de tool calls
- tokens consumidos
- tempo total
- autorrelato "concluído"
- uma única execução feliz

Combine evidências

- estado final verificado
- qualidade da trajetória
- latência e custo por sucesso
- segurança e intervenção humana
- repetições + dispersão
- revisão humana calibrada

Medir só resultado esconde fragilidade. Medir só trajetória recompensa atividade.

— ESTRATÉGIA DE AVALIAÇÃO

Três níveis de evidência

3 · Produção

tarefas reais, incidentes, custo, impacto e controle humano

realista

2 · Cenários

objetivo + sandbox + agente + transcript + rubrica + oráculo

causal

1 · Pré-flight

docs, comandos, schemas, testes, CI, segurança e recovery

barato

Pré-flight encontra lacunas antes de gastar tokens.

Cenário testa uma hipótese sob condições controladas.

Produção revela a distribuição e os riscos que o laboratório não cobre.

Controle a comparação antes de discutir o modelo

DESENHO

1. Fixe

tarefa, estado inicial, versões, orçamento, ferramentas e critério de sucesso.

EXECUÇÃO

2. Repita

3–5+ vezes, ordem alternada, sandbox limpo e captura integral da trajetória.

ANÁLISE

3. Triangule

oráculo determinístico, juiz calibrado e revisão humana de amostra.

Reporte a distribuição

— PROJETO ESCOLHIDO

AXIS

“Lighthouse para Agent Experience”: executa cenários, captura transcripts e produz um resultado composto.

Criado pela Netlify · licença MIT · clone fixado no commit 71fb758 · versão clonada 1.17.5.

Por que este?

- código, rubrica e roadmap abertos
- adapters para múltiplos agentes
- isolamento por workspace
- artefatos + transcript
- baseline e regressão em CI

Separar “funcionou?” de “deveria ter feito?”

40%

Goal achievement

O resultado atende aos checks ponderados da rubrica?

20%

Environment

Shell, arquivos, Git, build: sucesso + velocidade.

20%

Service

API, MCP e rede: sucesso + velocidade.

20%

Agent

Necessidade, relevância, peso e qualidade da decisão.

$\text{Result} = \text{goal} \times .4 + \text{environment} \times .2 + \text{service} \times .2 + \text{agent} \times .2$ · pesos configuráveis.

Mesmo bug. Mesmo teste. Contexto diferente.

BEFORE

Repo funcional, pouco orientador

Código e testes existem, mas o comando é legado; não há README, contrato nem instruções para agente.

AFTER

Mesmo sistema, affordances explícitas

Adiciona README, AGENTS.md, comandos canônicos, domínio de entradas e critério de verificação.

Hipótese: o contexto explícito reduz exploração e aumenta a verificação correta. Não presuma — repita e meça.

Prompt + estado + rubrica + artefatos

```
"prompt": "Resolva a issue #42: pricing  
produz valores fracionários ou negativos...",  
  
"setup": [{  
  "action": "run_script",  
  "command": "cp -R ... ."  
}],  
  
"artifacts": ["src/pricing.js"]
```

```
"judge": [  
  { "check": "retorna inteiro  
    e arredonda corretamente",  
    "weight": 0.25 },  
  { "check": "rejeita entradas  
    fora do domínio",  
    "weight": 0.25 },  
  { "check": "testes passaram  
    sem serem alterados",  
    "weight": 0.40 }  
]
```

Arquivos completos: demo/scenarios/01-before.json e 02-after.json.

— DEMO EM QUATRO COMANDOS

Clone validado → configuração → execução → evidência

• • •

```
cd opensource/axis
npm ci && npm run build && npm test && npm run typecheck

cd ../../demo
node validate-lab.mjs
node ../opensource/axis/dist/cli.js run --scenario 02-after
node ../opensource/axis/dist/cli.js reports latest --html
```

Autenticação: `codex login` ou `CODEX_API_KEY`.

Isolamento: workspace temporário por execução.

Plano B: pré-flight e relatório salvo; nunca dependa da rede ao vivo.

O contexto mudou mais a trajetória que o resultado

BEFORE · 98/100

Objetivo		100
Ambiente		93
Agente		95

115,7 s · 12 interações · 128.573 input tokens reportados

AFTER · 99/100

Objetivo		100
Ambiente		100
Agente		96

56,5 s · 9 interações · 68.819 input tokens reportados

Observado: after levou 51,2% menos tempo, 25% menos interações e 46,5% menos input tokens reportados. Com N=1, isso gera uma hipótese — não uma conclusão.

Pré-flight estrutural do AXIS



Contexto e descoberta		75
Ambiente reproduzível		100
Verificação e feedback		100
Automação e governança		50
Interfaces legíveis		100
Segurança e recuperação		100

Lacunas detectadas: guia de contribuição, arquivo SECURITY.md e ownership explícito. O scanner é heurístico; não é resultado AXIS.

— LIMITE EXPLÍCITO

88/100 ainda não responde se um agente conclui uma tarefa

O pré-flight responde

"Existem sinais estruturais para descobrir, executar e verificar o trabalho?"

O pré-flight não responde

"O agente tomou boas decisões, alcançou o estado correto e recuperou de falhas reais?"

"Prontidão é pré-condição. Resultado exige cenário."

Três gates antes de confiar no score

GATE 1

Segurança

Sem sandbox, privilégio mínimo, limites e controle de irreversíveis: **não execute autonomamente.**

GATE 2

Verificabilidade

Sem oráculo independente do agente: resultado **não conclusivo.**

GATE 3

Confiabilidade

Sem repetição, versões fixas e dispersão: comparação **não publicável.**

Regra de decisão: um risco crítico não é compensado pela média. Avalie dimensões e gates antes do total.

Transforme falha em backlog observável

1. Capture

transcript + estado + custo

2. Classifique

contexto, ambiente, serviço ou decisão?

3. Corrija

menor affordance que fecha o loop

4. Repita

mesmo protocolo e N execuções

5. Proteja

baseline + CI + owner

Baixo Environment?

setup, dependência, comando, latência.

Baixo Service?

schema, erro, rate limit, idempotência.

Baixo Agent?

descoberta, instrução, escopo, excesso de chamadas.

Um fluxo crítico em 30 dias

DIAS 1–5

Escolha

Tarefa de 15–30 min, frequente, reversível e com oráculo forte.

DIAS 6–12

Instrumente

Pré-flight, cenário, sandbox, transcript e baseline inicial.

DIAS 13–20

Melhore

Corrija 1–3 gargalos de contexto, feedback ou interface.

DIAS 21–30

Proteja

Repita, calibre, documente owner e automatize regressão.

AX madura quando a autonomia encontra evidência

1

Projete o loop

descoberta, ação, feedback, verificação e recuperação.

2

Meça camadas

pré-flight + cenários + produção; nunca uma nota solitária.

3

Use gates

segurança, verificabilidade e confiabilidade antes da média.

Para aprofundar e auditar

1. [ISO 9241-210:2019](#) — human-centred design.
2. [ISO 9241-11:2018](#) — usability.
3. [Forsgren et al., SPACE](#) — ACM Queue, 2021.
4. [DORA 2025](#) — AI-assisted software development.
5. [AgentExperience.ax](#) — definição comunitária.
6. [Netlify: Agent Experience](#) — enquadramento.
7. [AXIS](#) e [scoring](#) — documentação oficial.
8. [netlify/axis](#) — código MIT usado na demo.
9. [AGENTS.md](#) — formato aberto para contexto.
10. [Linux Foundation / AAIL](#) — governança aberta.
11. [MCP Specification](#) — tools/resources/prompts.
12. [NIST AI RMF](#) — governança e risco.
13. [OWASP Agentic Top 10 2026](#) — segurança.

Bibliografia comentada, limitações e datas de consulta: `fontes.md`.

Uma página para a retrospectiva

Resultados

- taxa de sucesso / N
- mediana + dispersão de tempo
- tokens/custo por sucesso
- intervenções e retries
- falhas por etapa do loop

Contexto controlado

- cenário + hash
- repo + commit
- agente/modelo/esforço
- juiz/modelo/rubrica
- ambiente + ferramentas

Decisão: manter baseline · abrir melhoria · reduzir autonomia · bloquear cenário · revisar rubrica.

Modelo preenchível em `demo/evidencias/RELATORIO-EXEMPLO.md`.